La Plateforme

La grande école du numérique pour tous

Système, Scripts et Sécurité

Un voyage à travers les commandes, scripts et interactions avancées

Debian | Bash Scripting | Sécurité | API | Processus

Gabin • Mai 2026

Sommaire

01 **Commandes de recherche avancée**
grep, find, pipes

02 **Compression & Décompression**
tar, gzip, bzip2

03 **Manipulation de texte – AWK**
CSV & extraction de données

04 **Surveillance des ressources**
ps, top, vmstat, df

05 **Scripting avancé – Sauvegarde**
Automatisation & historique

06 **Mises à jour & Dépendances**
apt, scripts d'installation

07 **Utilisation d'une API Web**
curl, JSON, logging, sécurité

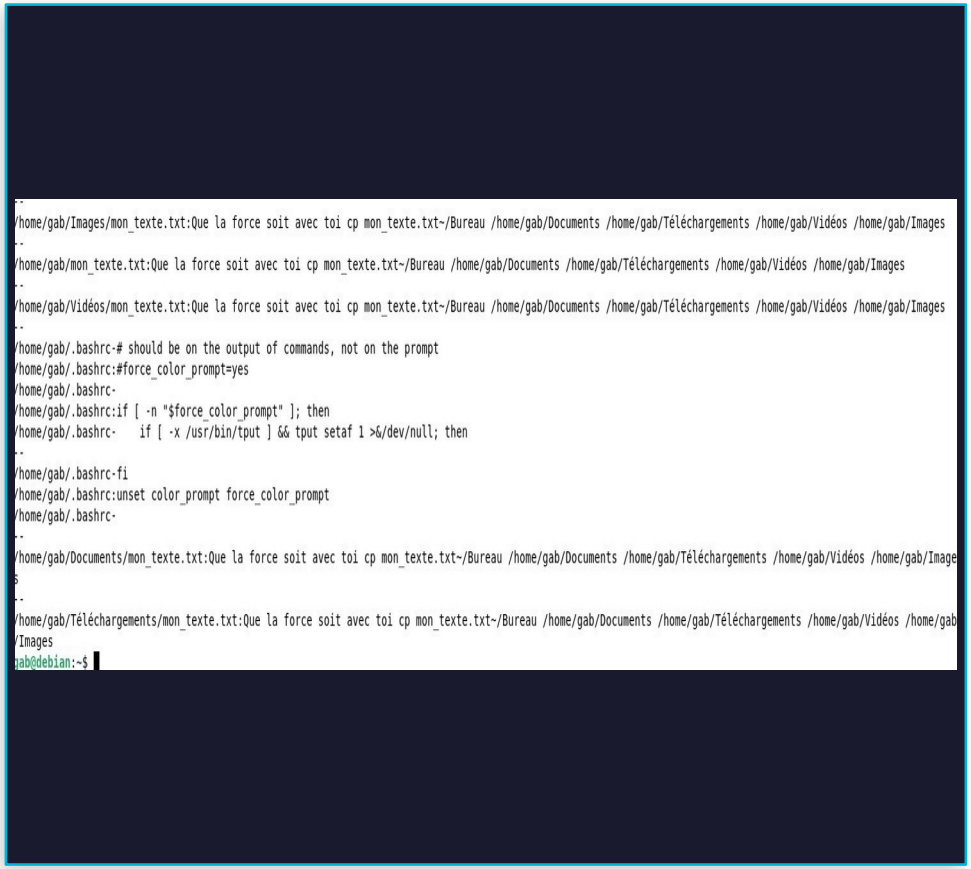
01 Recherche avancée – création et copie des fichiers

- echo "Que la force soit avec toi" > mon_texte.txt : crée le fichier avec redirection >
- cp mon_texte.txt ~/Bureau ~/Documents ~/Téléchargements ~/Vidéos ~/Images : copie vers 5 dossiers en une commande
- ls vérifie la présence du fichier dans le répertoire courant



01 Résultats grep – localisation des 5 fichiers

- grep -r "force" ~ : recherche récursive du mot « force » depuis le home (~)
- Chaque résultat : chemin/fichier:contenu de la ligne correspondante
- 5 fichiers retrouvés : Bureau, Documents, Vidéos, Images, Téléchargements
- .bashrc remonté aussi : grep cherche dans TOUS les fichiers du home
- Astuce : ajouter --include="*.txt" pour filtrer uniquement les .txt



02 Création du répertoire Plateforme

- cp ~/Documents/mon_texte.txt ~/Documents/Plateforme/ : copie du fichier source
- cd Documents/ puis cd Plateforme/ : navigation dans le dossier créé
- ls confirme la présence de mon_texte.txt dans Plateforme/



02 Duplication – 5 fichiers dans Plateforme

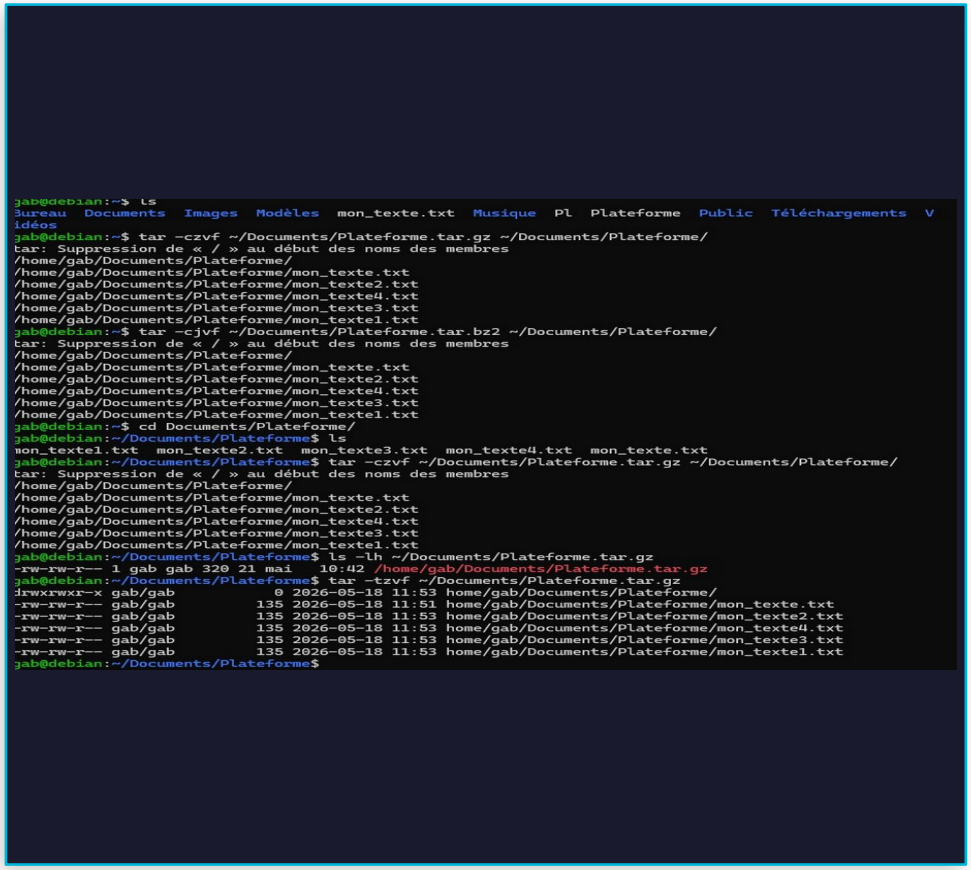
- 4 commandes cp successives : mon_texte1.txt → mon_texte4.txt
- Total : 5 fichiers (original + 4 copies) dans ~/Documents/Plateforme/
- ls confirme : mon_texte1 à mon_texte4 + mon_texte.txt



cp ~/Documents/mon_texte.txt ~/Documents/Plateforme/mon_texteN.txt

02 Archivage tar.gz et tar.bz2 + inspection

- tar -czvf : créer (-c), verbose (-v), compression gzip (-z), fichier (-f)
- tar -cjvf : même chose avec bzip2 (-j) → meilleur ratio, plus lent
- tar -tzvf : liste le contenu d'une archive sans l'extraire
- ls -lh : taille de l'archive (320 octets ici)
- « Suppression de / » : tar retire le slash absolu du chemin (comportement normal)



02 Décompression tar.gz et tar.bz2

- tar -xzvf : extraire (-x) + gzip (-z) + verbeux (-v) vers un dossier (-C)
- tar -xjvf : même chose pour bzip2 (-j)
- -C ~/Documents/ : spécifie le répertoire de destination
- Les fichiers sont restaurés avec leur chemin relatif d'origine
- tar -xvf (sans -z ni -j) : détection automatique de la compression

```
gab@debian:~/Documents/Plateforme$ tar -xzvf ~/Documents/Plateforme.tar.gz -C ~/Documents/
home/gab/Documents/Plateforme/
home/gab/Documents/Plateforme/mon_texte.txt
home/gab/Documents/Plateforme/mon_texte2.txt
home/gab/Documents/Plateforme/mon_texte4.txt
home/gab/Documents/Plateforme/mon_texte3.txt
home/gab/Documents/Plateforme/mon_texte1.txt
gab@debian:~/Documents/Plateforme$ tar -xjvf ~/Documents/Plateforme.tar.bz2 -C ~/Documents/
home/gab/Documents/Plateforme/
home/gab/Documents/Plateforme/mon_texte.txt
home/gab/Documents/Plateforme/mon_texte2.txt
home/gab/Documents/Plateforme/mon_texte4.txt
home/gab/Documents/Plateforme/mon_texte3.txt
home/gab/Documents/Plateforme/mon_texte1.txt
gab@debian:~/Documents/Plateforme$
```

tar -xzvf archive.tar.gz -C ~/destination/ | tar -xjvf archive.tar.bz2 -C ~/destination/

03 Script Python → CSV + extraction AWK

- Script Python créé inline avec cat + heredoc (EOF) directement dans le terminal
- Module csv + writer.writerows() : écrit l'en-tête et les 4 lignes de données
- python3 create_csv.py génère personnes.csv avec succès
- awk -F',' : virgule comme séparateur de champ
- 'NR>1 {print $3}' : ignore la ligne header (NR>1) et affiche le 3ème champ (ville)
- Résultat : Paris, Lyon, Marseille, Toulouse

```
gab@debian:~/Documents/Plateforme$ cat > create_csv.py << 'EOF'
import csv

data = [
    ["Nom", "Age", "Ville"],
    ["Jean", "25", "Paris"],
    ["Marie", "30", "Lyon"],
    ["Pierre", "22", "Marseille"],
    ["Sophie", "35", "Toulouse"]
]

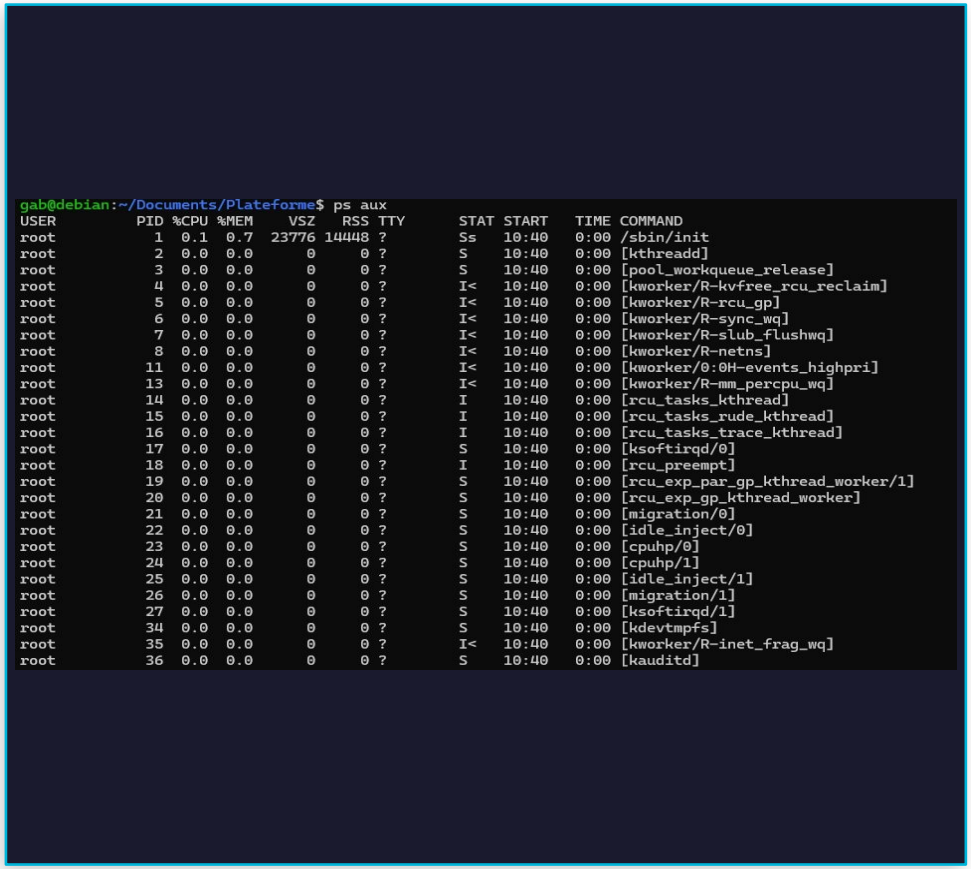
with open("personnes.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)

print("Fichier CSV créé avec succès.")
EOF
gab@debian:~/Documents/Plateforme$ python3 create_csv.py
Fichier CSV créé avec succès.
gab@debian:~/Documents/Plateforme$ awk -F',' 'NR>1 {print $3}' personnes.csv
Paris
Lyon
Marseille
Toulouse
```

awk -F',' 'NR>1 {print $3}' personnes.csv → affiche toutes les villes

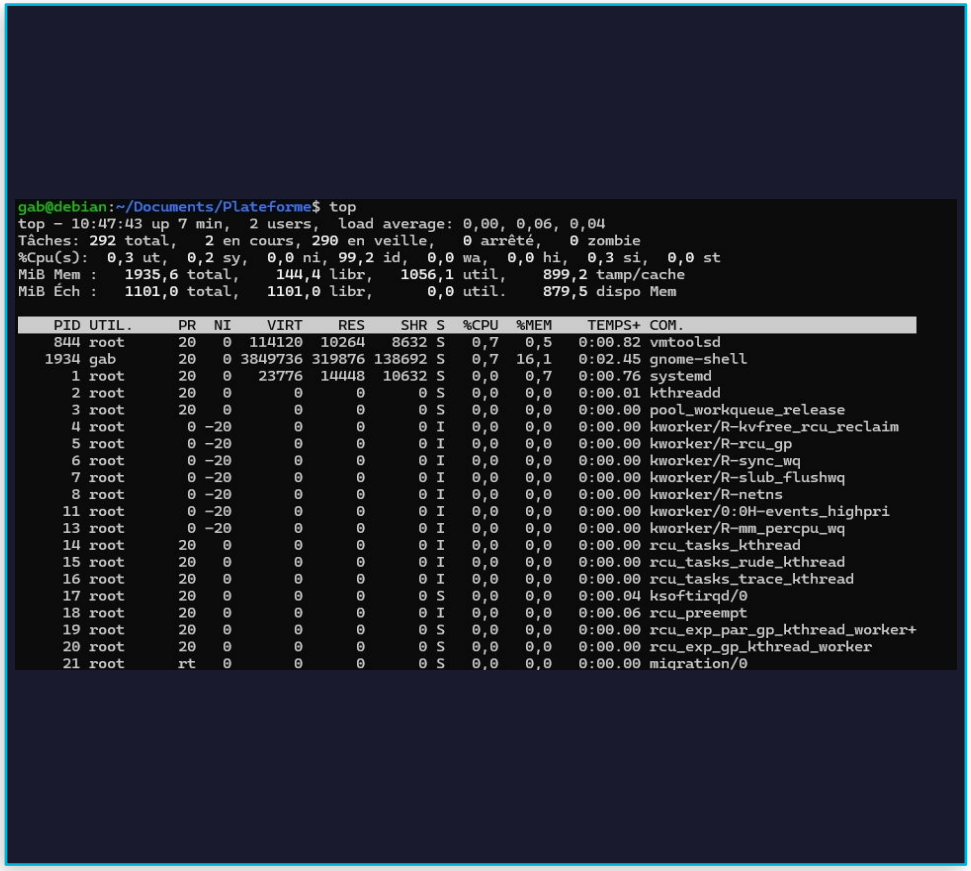
04 Surveillance – ps aux

- ps aux : tous les processus (a=tous users, u=format user, x=sans TTY)
- Colonnes : USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
- PID 1 = /sbin/init (systemd) : 1er processus lancé au boot
- Crochets [kthreadd]… : threads noyau kernel (pas de processus user)
- kill PID → signal SIGTERM (15) : demande d'arrêt propre
- kill -9 PID → signal SIGKILL (9) : terminaison forcée, non ignorable



04 Surveillance temps réel – top

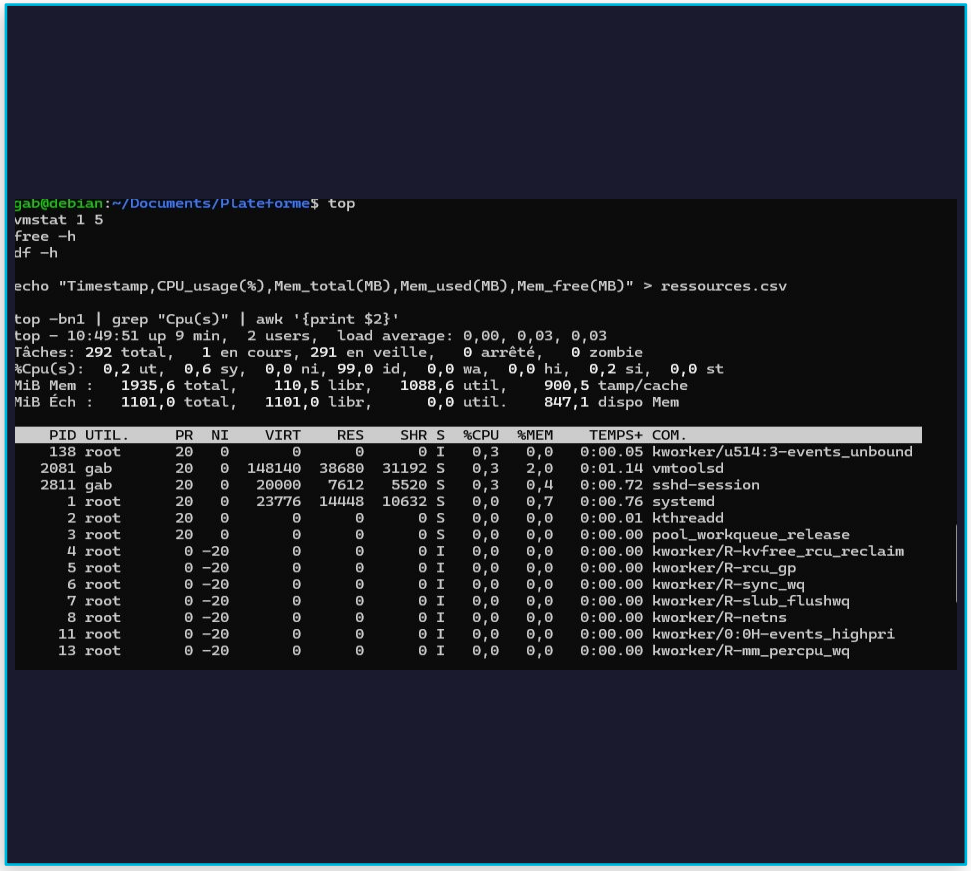
- top : processus triés par %CPU, rafraîchissement en temps réel
- Load average : charge sur 1 / 5 / 15 minutes (0,00 = système très libre)
- %Cpu id=99,2% : CPU quasi inactif | gnome-shell consomme 16,1% RAM
- MiB Mem 1935 total | 144 libre | 1056 utilisé | 899 cache
- Touches utiles : q=quitter, k=tuer un processus, M=trier par mémoire



top | q pour quitter | k PID pour tuer | M pour trier par mémoire

04 vmstat, free, df + export CSV

- vmstat 1 5 : stats mémoire/CPU toutes les 1s pendant 5 secondes
- free -h : mémoire totale / libre / utilisée en format lisible humain
- df -h : espace disque par partition
- top -bn1 | grep 'Cpu(s)' | awk : extrait le %CPU en mode batch (non-interactif)
- echo … > ressources.csv crée le header | >> ajoute les données (append)



05 Script surveillance.sh – collecte périodique

- Boucle for i in $(seq 1 5) : 5 itérations avec sleep 2 entre chaque
- TIMESTAMP=$(date …) : horodatage AAAA-MM-JJ HH:MM:SS
- CPU via top -bn1 | grep + awk + tr pour supprimer les virgules/espaces
- MEM via free -m | awk NR==2 : total, utilisé, libre en MB sur la 2ème ligne
- chmod +x : rend le script exécutable | ./surveillance.sh : lance le script

```
gab@debian:~/Documents/Plateforme$ cat > surveillance.sh << 'EOF'
#!/bin/bash

OUTPUT="ressources.csv"
echo "Timestamp,CPU(%),Mem_Total(MB),Mem_Used(MB),Mem_Libre(MB)" > "$OUTPUT"

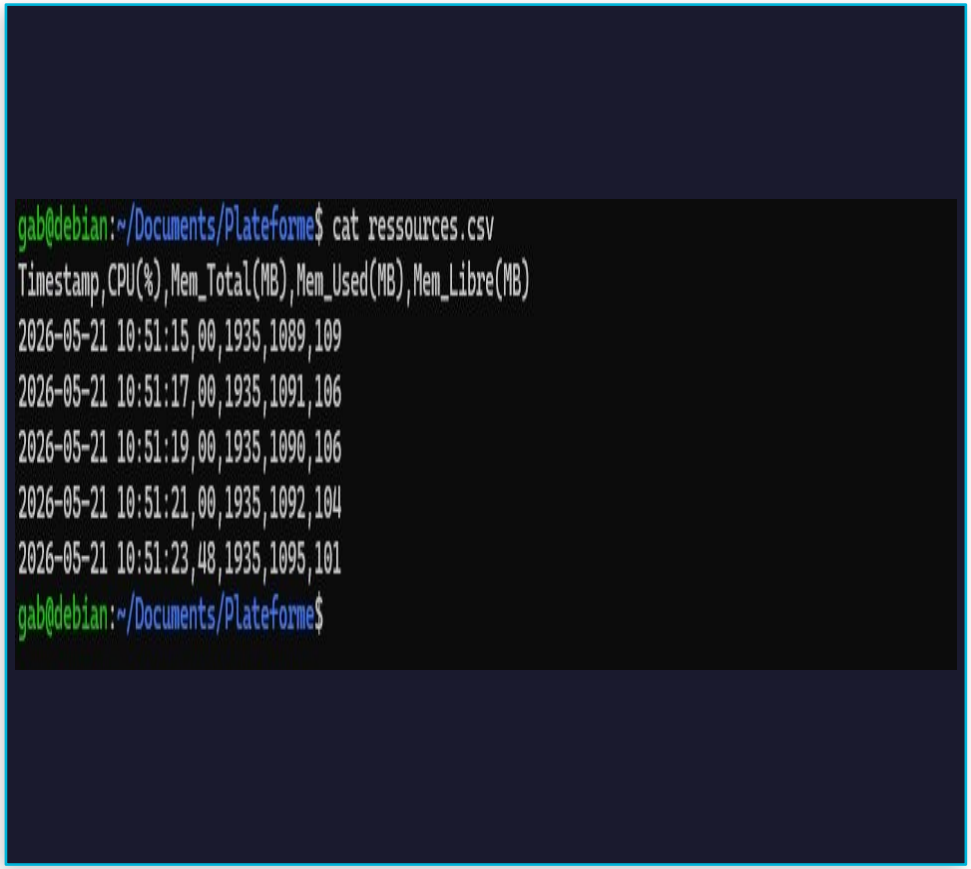
for i in $(seq 1 5); do
  TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
  CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | tr -d '%us,')
  MEM=$(free -m | awk 'NR==2{printf "%s,%s,%s", $2,$3,$4}')
  echo "$TIMESTAMP,$CPU,$MEM" >> "$OUTPUT"
  sleep 2
done

echo "Surveillance enregistrée dans $OUTPUT"
EOF
gab@debian:~/Documents/Plateforme$ chmod +x surveillance.sh
gab@debian:~/Documents/Plateforme$ ./surveillance.sh
Surveillance enregistrée dans ressources.csv
gab@debian:~/Documents/Plateforme$
```

#!/bin/bash | chmod +x script.sh | ./script.sh pour exécuter

05 Résultat – ressources.csv généré

- cat ressources.csv : affiche les 5 lignes collectées + le header
- Colonnes : Timestamp, CPU(%), Mem_Total(MB), Mem_Used(MB), Mem_Libre(MB)
- CPU stable à 0,00% sauf dernière mesure à 0,48% (pic momentané)
- RAM utilisée : ~1090 MB sur 1935 MB totaux | ~100 MB libre
- Fichier CSV prêt à analyser avec Python/pandas ou Excel



Données exploitables avec pandas, Excel ou tout outil tableur

05 Script sauvegarde.sh – backup automatisé

- Variables SOURCE, BACKUP_DIR, LOG_FILE définies en haut du script
- mkdir -p : crée le dossier de sauvegardes s'il n'existe pas déjà
- ARCHIVE = nom unique avec horodatage via $(date ...)
- tar -czf crée l'archive | $? -eq 0 vérifie le code retour (0 = succès)
- >> LOG_FILE : ajoute une entrée à l'historique (append, pas d'écrasement)
- ls -t | tail -n +6 | xargs rm : rotation – garde les 5 dernières sauvegardes

```
gab@debian:~/Documents/Plateforme$ cat > sauvegarde.sh << 'EOF'
#!/bin/bash

SOURCE=~/Documents/Plateforme
BACKUP_DIR=~/sauvegardes
LOG_FILE=~/sauvegardes/historique.log

mkdir -p "$BACKUP_DIR"

DATE=$(date "+%Y%m%d_%H%M%S")
ARCHIVE="$BACKUP_DIR/Plateforme_$DATE.tar.gz"

tar -czf "$ARCHIVE" "$SOURCE"

if [ $? -eq 0 ]; then
  echo "[$DATE] Sauvegarde réussie : $ARCHIVE" >> "$LOG_FILE"
  echo "Sauvegarde créée : $ARCHIVE"
else
  echo "[$DATE] ERREUR lors de la sauvegarde" >> "$LOG_FILE"
  echo "Erreur !"
fi

ls -t "$BACKUP_DIR"/Plateforme_*.tar.gz | tail -n +6 | xargs -r rm --
echo "Historique des sauvegardes :"
cat "$LOG_FILE"
EOF
gab@debian:~/Documents/Plateforme$ chmod +x sauvegarde.sh
gab@debian:~/Documents/Plateforme$ ./sauvegarde.sh
tar: Suppression de « / » au début des noms des membres
Sauvegarde créée : /home/gab/sauvegardes/Plateforme_20260521_105313.tar.gz
Historique des sauvegardes :
[20260521_105313] Sauvegarde réussie : /home/gab/sauvegardes/Plateforme_20260521_105313.tar.gz
gab@debian:~/Documents/Plateforme$
```

Rotation automatique : les archives > 5 sont supprimées

Script mises_a_jour.sh – mise à jour interactive

- sudo apt update : rafraîchit la liste des paquets disponibles
- apt list --upgradable 2>/dev/null | grep -c upgradable : compte les MAJ
- read -p : demande confirmation à l'utilisateur avant d'agir
- sudo apt upgrade -y : installe les mises à jour si l'utilisateur répond « o »
- Structure if/else/fi : 3 cas → MAJ installées / annulées / système déjà à jour

```
gab@debian:~/Documents/Plateforme$ cat > mises_a_jour.sh << 'EOF'
#!/bin/bash

echo "=== Recherche de mises à jour ==="
sudo apt update

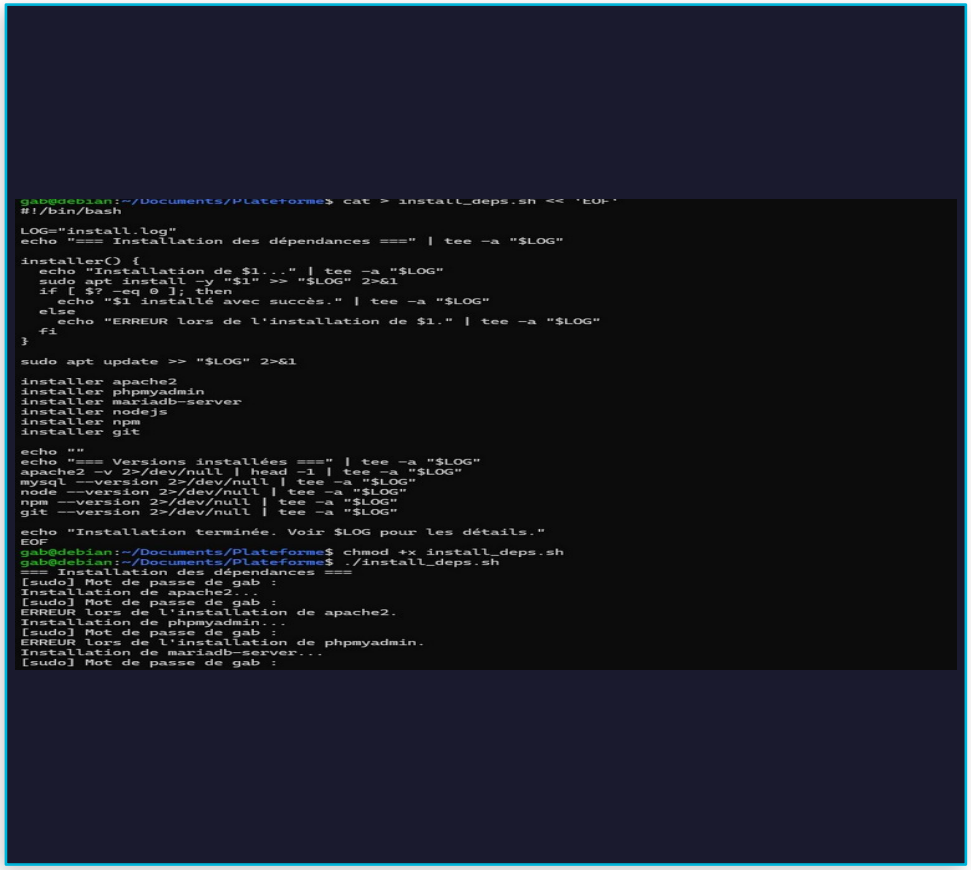
UPDATES=$(apt list --upgradable 2>/dev/null | grep -c upgradable)
echo "$UPDATES paquet(s) disponible(s)."

if [ "$UPDATES" -gt 0 ]; then
  apt list --upgradable 2>/dev/null
  read -p "Voulez-vous installer les mises à jour ? (o/n) : " REPONSE
  if [ "$REPONSE" = "o" ]; then
    sudo apt upgrade -y
    echo "Mises à jour installées."
  else
    echo "Mises à jour annulées."
  fi
else
  echo "Système déjà à jour."
fi
EOF
gab@debian:~/Documents/Plateforme$ chmod +x mises_a_jour.sh
gab@debian:~/Documents/Plateforme$ ./mises_a_jour.sh
=== Recherche de mises à jour ===
[sudo] Mot de passe de gab :
```

apt update | apt list --upgradable | apt upgrade -y

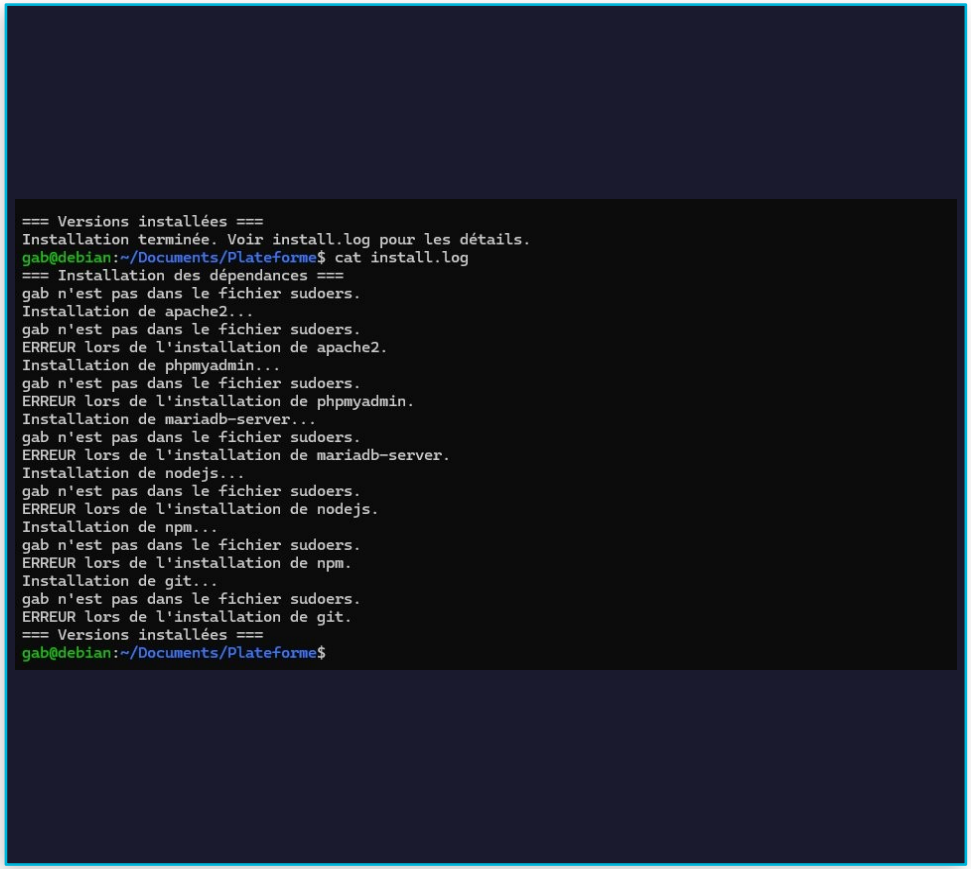
06 Script install_deps.sh – stack web

- Fonction installer() réutilisable : installe + log le résultat de chaque paquet
- tee -a $LOG : affiche ET écrit dans le journal simultanément
- 2>&1 : redirige stderr vers stdout pour tout capturer dans le log
- $? -eq 0 : vérifie si l'installation a réussi, log le statut en conséquence
- Installe : apache2, phpmyadmin, mariadb-server, nodejs, npm, git
- Affiche les versions installées à la fin (apache2 -v, mysql, node, npm, git)



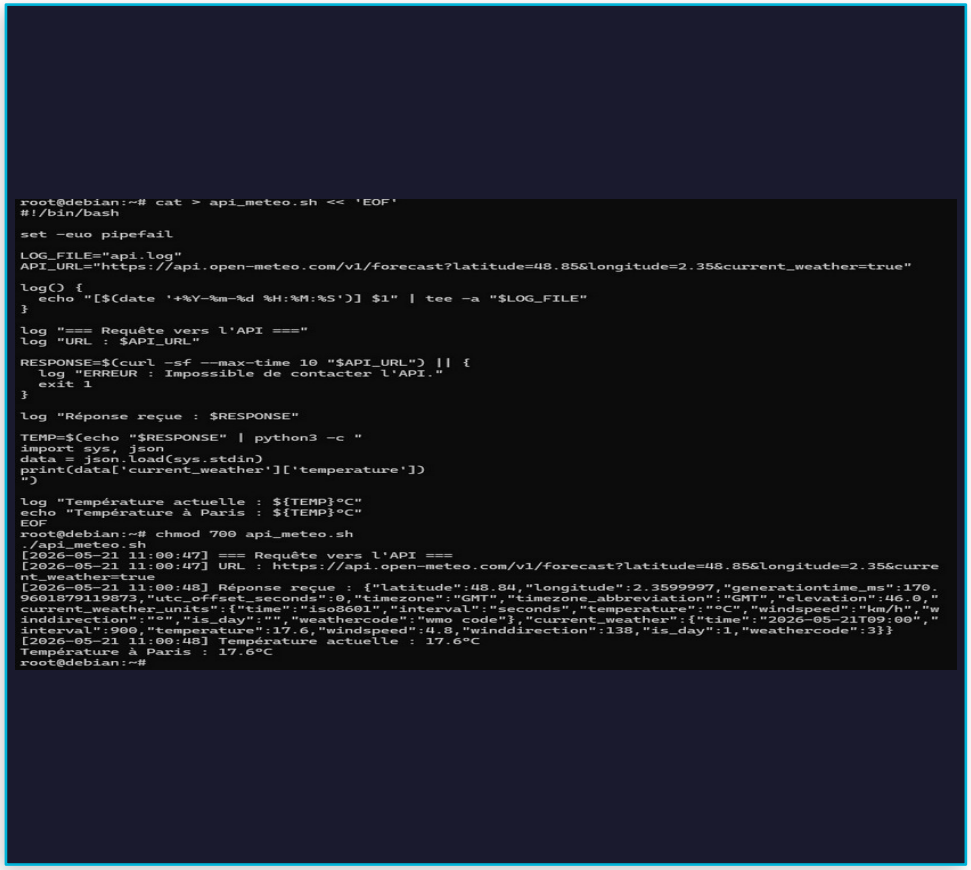
Résultat – install.log & erreurs sudoers

- Erreur : « gab n'est pas dans le fichier sudoers » → sudo refusé pour toutes les installations
- Solution : sudo usermod -aG sudo gab (en tant que root)
- Puis se déconnecter / reconnecter pour que le changement soit pris en compte
- Alternative : éditer /etc/sudoers avec visudo et ajouter gab ALL=(ALL) ALL
- Le log capture toutes les erreurs : pratique pour diagnostiquer à distance



07 Script api_meteo.sh – Open-Meteo + logging sécurisé

- set -euo pipefail : stoppe le script dès la 1ère erreur (bonne pratique sécurité)
- curl -sf --max-time 10 : requête HTTPS avec timeout 10s, silencieux en cas d'erreur
- || { log "ERREUR" ; exit 1 } : gestion d'erreur si l'API est injoignable
- python3 -c : parse le JSON inline pour extraire la température actuelle
- log() : chaque requête ET réponse horodatées dans api.log (journalisation)
- chmod 700 : permissions restrictives – seul le propriétaire peut exécuter



API Open-Meteo (HTTPS gratuite) | Résultat : 17,6°C à Paris

Compétences acquises

🔍 Recherche avancée

grep, find, pipes, redirections

📦 Archivage

tar, gzip, bzip2, compression/décompression

📊 Manipulation données

awk, CSV, Python scripting

⚙️ Processus

ps, top, kill, SIGTERM vs SIGKILL

📈 Ressources système

vmstat, free, df, surveillance CSV

💾 Sauvegarde automatisée

Script bash, rotation, historique log

🔄 Mises à jour & deps

apt, gestion dépendances web stack

🌐 API & Sécurité

curl, HTTPS, logging, set -euo pipefail